

8-step Python Context Managers Exercises

What you will explore

Complete these exercises sequentially to review scoping, resource cleanup, class-based protocols, exception handling, and real-world pipelines.

Exercise 1: Logging Context Protocol (Scope Warm-up)

Objective: Build a minimal class-based context manager that outputs log notifications on entering and exiting code blocks.

Concept: The context manager protocol uses `__enter__` and `__exit__` to establish operational boundaries. When an exception occurs, the exit hook is notified and must return `False` to allow standard error propagation.

```
● ● ● exercise_1_protocol_template.py

class LoggingContext:
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        # TODO: Print entry message and return self
        pass

    def __exit__(self, exc_type, exc_val, exc_tb):
        # TODO: Print exit message noting exception status
        # TODO: Return False to let any errors propagate
        pass
```

Expected Output:

```
[LoggingContext] Entering block: TaskA
    Inside active context task block
[LoggingContext] Exiting block: TaskA successfully
```

Exercise 2: Safe File Manager Class (Resource Safety)

Objective: Implement a robust, class-based custom file writer that guarantees the file handle is closed on context exit.

Concept: Encapsulating raw file descriptors within classes allows `__exit__` to act as an automated garbage collector, guaranteeing system handle closure even if operations inside the block raise exceptions.

```
● ● ● exercise_2_file_manager_template.py

class SafeFileManager:
    def __init__(self, filepath, mode):
        self.filepath = filepath
        self.mode = mode
        self.file = None

    def __enter__(self):
        # TODO: Open file handle in specified mode and return it
        pass

    def __exit__(self, exc_type, exc_val, exc_tb):
        # TODO: Safely close file handle if open
        pass
```

Expected Output:

Write complete.

Read verification: Writing safely through a class-based context manager.

Exercise 3: Exception-Aware SQLite Transaction Manager

Objective: Build a transaction context manager that dynamically commits database insertions on success or rolls them back on exceptions.

Concept: Monitoring the `exc_type` argument inside `__exit__` lets managers decide whether to commit or abort operations, ensuring relational data integrity.

```
● ● ● exercise_3_transaction_template.py

class SafeTransaction:
    def __init__(self, db_path):
        self.db_path = db_path
        self.conn = None

    def __enter__(self):
        # TODO: Open SQLite connection and return connection object
        pass

    def __exit__(self, exc_type, exc_val, exc_tb):
        # TODO: Commit if exc_type is None; otherwise rollback
        # TODO: Close connection always
        pass
```

Expected Output:

Running Test Case 1 (Commit):

Inserted records inside success block

[SafeTransaction] Database updates committed successfully.

Exercise 4: Reusable Block Timer Profiler (Metrics)

Objective: Build a class-based Timer context manager that profiles execution durations and stores metrics as public attributes.

Concept: `__enter__` captures the start timestamp using `time.perf_counter()`. Upon exit, the elapsed duration is calculated and made accessible as an attribute for external analytics.

```
● ● ● exercise_4_timer_template.py

import time

class BlockTimer:
    def __init__(self, label="Block"):
        self.label = label
        self.start = None
        self.elapsed = None

    def __enter__(self):
        # TODO: Record start time using time.perf_counter() and return self
        pass

    def __exit__(self, exc_type, exc_val, exc_tb):
        # TODO: Calculate duration and store in self.elapsed
        pass
```

Expected Output:

[Large Computation Loop] Elapsed: 0.123456 seconds

Outer access verification - Elapsed: 0.123456 seconds

Exercise 5: Generator-Based FileManager (Generators)

Objective: Rebuild Exercise 2's SafeFileManager utilizing the standard @contextmanager decorator.

Concept: Using contextlib allows writing concise setup-teardown flows. Wrapping the yield statement in try-finally guarantees that the teardown code executes even if the nested code block fails.

```
● ● ● exercise_5_generator_manager_template.py

from contextlib import contextmanager

@contextmanager
def safe_file_manager(filepath, mode):
    # TODO: Open file handle
    # TODO: Yield inside a try block, close inside finally
    pass
```

Expected Output:

Generator write complete.

Read verification: Writing safely through a generator-based manager.

Exercise 6: Isolated Temporary Workspace (Temporary Files)

Objective: Create a context manager that spins up a secure temporary directory, yields its `pathlib.Path`, and deletes it on `exit`.

Concept: Creating sandboxed directories and guaranteeing their removal protects local filesystems from garbage pollution and metadata leakage.

```
● ● ● exercise_6_temp_workspace_template.py

from pathlib import Path
from tempfile import TemporaryDirectory
from contextlib import contextmanager

@contextmanager
def temporary_workspace():
    # TODO: Create TemporaryDirectory
    # TODO: Yield Path object and guarantee auto-cleanup
    pass
```

Expected Output:

Workspace path active: /tmp/tmp_dir_path

Does workspace still exist on disk? False

Exercise 7: Dynamic Multiple File Handles with ExitStack

Objective: Write a function that accepts a list of file paths and safely writes a matching header line to all of them using ExitStack.

Concept: Opening resource lists dynamically without nesting multiple with blocks requires ExitStack. If opening file N fails, ExitStack automatically unwinds and closes files 1 to N-1 safely.

```
● ● ● exercise_7_exit_stack_template.py

from contextlib import ExitStack

def write_headers_to_all(filepaths, header_text):
    # TODO: Open dynamic list of files using stack.enter_context
    # TODO: Write header_text to all handles
    pass
```

Expected Output:

Batch headers written.

test1.txt contents: 'PROPRIETARY AND CONFIDENTIAL TRAINING RESOURCE'

Exercise 8: Capstone Data Processing Pipeline (ETL)

Objective: Design a timed, exception-safe data processing pipeline that stages records in a temporary directory before loading them inside a relational database transaction.

Concept: By stacking custom timers, temporary directory sandboxes, and database transactions, you build a robust production pipeline. If the database load fails, the transaction is rolled back, but the temporary files are still safely unlinked and the execution duration is logged.

```
● ● ● exercise_8_capstone_pipeline_template.py

# Task: Combine BlockTimer, SafeTransaction, and temporary_workspace.
# Staging records into a temporary CSV file.
# Process records inside a database transaction, timing the entire operation.

def run_data_pipeline(input_records, db_path):
    # TODO: Implement full timed staging and insertion transaction pipeline
    pass
```

Expected Output:

Running capstone data pipeline...

[SafeTransaction] Database operations committed.

[Pipeline ETL Execution] Process duration: 0.015000 seconds